Social Discounting Package

Naomi Nero & Abigail Marsh

2026-11-12

Version 2.0.0

Correspondence concerning this manual and package should be addressed to Naomi Nero, Department of Psychology, Georgetown University, White Gravenor Hall, Washington, DC 20057, United States.

Email: nn444@georgetown.edu (mailto:nn444@georgetown.edu)

This script cleans and analyzes the Original Social Discounting Task (Long Form) data and Social Discounting Task Short-Form exported. Data can be collected using the provided Qualtrics templates or the Shiny App code, both of which save the data and variable names in a compatible format to be analyses with this script. Further explanation of the data preparation, cleaning, and analysis steps are explains in the accompanying manual.

Set up

This section loads all packages required for data cleaning, reshaping, and modeling.

User Inputs

It assumes data were exported from Qualtrics as a CSV file using **Export values** (not Export labels).

Modify your paths and filenames for your dataset.

- path: folder containing the Qualtrics CSV files
- sd_long: filename for the long-form task data
- sd_short: filename for the short-form task data

Model Fitting Functions

Nonlinear mixed-effects models (`nlme`) are estimated iteratively, starting from initial guesses (“starting values”) for each parameter. If the starting values are far from the final estimates, the model can fail to converge or settle on a poorer solution. Rather than trying different starting values by hand, the functions below automate this step and the model comparison step. They are used for every model in this script.

Control settings (`ctl`)

All `nlme` models use the same control settings, which allow more iterations than the defaults before the model stops.

`fit_best_start()`

This function:

- fits the model once for every combination of starting values in a grid
- skips any combination where the model fails to converge
- keeps the converged model with the highest log-likelihood (the best-fitting solution)
- prints how many of the starting values converged

Arguments:

- `fit_fun` : a function that takes a vector of starting values and returns an `nlme` model
- `start_grid` : a data frame of starting values to try, with one column per parameter (e.g., v0 and logk), created with `expand.grid()`
- `n_extra` : the number of predictors in the model (Model 2 only). The starting values for the predictor slopes are set to 0.

A table of every starting value tried, and the estimates it reached, is saved with the model. You can view it with `attr(model, "start_search")` :

- If the converged fits reach the same log-likelihood and the same estimates, the solution is stable and does not depend on the starting values.
- If they reach different estimates with similar log-likelihoods, the model may be poorly identified and should be simplified.
- If only a few starting values converge, interpret the results with caution. If none converge, widen `start_grid` or simplify the model.

`aic_table()`

This function takes a named list of models and returns a model comparison table (see the Model Comparison sections below). It gives the same values as `aictab()` from the AICcmodavg package, but it also works when `nlme` models are compared with the `lme` linear model, which `aictab()` does not allow.

```
# Control settings used for all nlme models
ctl <- nlmeControl(msMaxIter = 1000, maxIter = 500, pnlsMaxIter = 50)

# Try a grid of starting values and keep the best-fitting model that converges
fit_best_start <- function(fit_fun, start_grid, n_extra = 0) {
  p <- ncol(start_grid)

  results <- purrr::map(seq_len(nrow(start_grid)), function(i) {
    st  <- c(unlist(start_grid[i, ]), rep(0, n_extra))
    fit <- tryCatch(fit_fun(st), error = function(e) NULL)  # failed fits return NULL
    list(start = st, fit = fit)
  })

  ok <- purrr::keep(results, ~ !is.null(.x$fit))
  if (length(ok) == 0) stop("No starting values converged. Widen start_grid or simplify
the model.")

  search <- purrr::map_dfr(ok, function(r) data.frame(
    as.list(setNames(r$start[1:p], paste0("start_", names(start_grid)))),
    logLik = as.numeric(logLik(r$fit)),
    as.list(setNames(fixef(r$fit)[1:p], paste0("est_", names(start_grid)))),
    check.names = FALSE))

  best <- ok[[which.max(search$logLik)]]
  message(length(ok), " of ", nrow(start_grid), " starting values converged.")
  attr(best$fit, "start_search") <- search
  best$fit
}

# Model comparison table: K, log-likelihood, AICc, delta AICc, AICc weights
aic_table <- function(models) {
  tab <- data.frame(
    Model = names(models),
    K     = sapply(models, function(m) attr(logLik(m), "df")),  # number of parameters
    LL    = sapply(models, function(m) as.numeric(logLik(m))),  # log-likelihood
    n     = sapply(models, nobs)                                # number of observations
  )
  tab$AICc   <- -2 * tab$LL + 2 * tab$K + (2 * tab$K * (tab$K + 1)) / (tab$n - tab$K -
1)
  tab$Delta  <- tab$AICc - min(tab$AICc)
  tab$Weight <- exp(-0.5 * tab$Delta) / sum(exp(-0.5 * tab$Delta))
  tab <- tab[order(tab$AICc), ]
  tab$Cum_Weight <- cumsum(tab$Weight)
  rownames(tab) <- NULL
  tab
}
```

SD LONG FORM

This is the data cleaning and analysis pipeline for the original Social Discounting Task

Import Data

This section imports the Qualtrics data file and cleans the data so the SD columns and subnum columns are numeric. It also drops the first 2 header rows seen in a qualtrics output

```
data_wide <- read.delim(
  file = paste0(path, sd_long),
  sep = ",",
  quote = "\"",
  header = TRUE
)

sd_cols <- grep("^sd\\d+_\\d$", names(data_wide), value = TRUE)

data_wide <- data_wide %>%
  mutate(across(all_of(sd_cols), ~ suppressWarnings(as.numeric(.x)))) %>%
  filter(if_any(all_of(sd_cols), ~ !is.na(.x))) %>%   # drops the 2 Qualtrics header row
s
  mutate(subnum = type.convert(na_if(trimws(as.character(subnum)), ""), as.is = TRUE))
# numeric IDs stay numeric (done after the header rows are removed)
```

Expected csv variable names

This analysis script assumes that the csv export contains the following variables.

Subject identifier

- subnum: unique participant ID (participants without a value in subnum will be removed during data cleaning)

Social Discounting Trial name structure

Each trial is named using the following structure: sd[distance]_[trial]

Here is a complete list of required variables

- sd1_1 – sd1_9
- sd2_1 – sd2_9
- sd5_1 – sd5_9
- sd10_1 – sd10_9
- sd20_1 – sd20_9
- sd50_1 – sd50_9
- sd100_1 – sd100_9

Response coding

Responses should be exported from Qualtrics using Export values. For trials 1-8, expected coding is:

- 0 = kept money
- Positive value (80-10) = shared money (amount offered to keep - $75)

On trial 9 (keep 75 or split 75/75), the expected coding is:

- -5 = kept money

- 0 = shared money

Check to make sure your data has all of the needed column/column names

```
names(data_wide)
```

```
##  [1] "subnum"  "sd1_1"   "sd1_2"   "sd1_3"   "sd1_4"   "sd1_5"   "sd1_6"
##  [8] "sd1_7"   "sd1_8"   "sd1_9"   "sd2_1"   "sd2_2"   "sd2_3"   "sd2_4"
## [15] "sd2_5"   "sd2_6"   "sd2_7"   "sd2_8"   "sd2_9"   "sd5_1"   "sd5_2"
## [22] "sd5_3"   "sd5_4"   "sd5_5"   "sd5_6"   "sd5_7"   "sd5_8"   "sd5_9"
## [29] "sd10_1"  "sd10_2"  "sd10_3"  "sd10_4"  "sd10_5"  "sd10_6"  "sd10_7"
## [36] "sd10_8"  "sd10_9"  "sd20_1"  "sd20_2"  "sd20_3"  "sd20_4"  "sd20_5"
## [43] "sd20_6"  "sd20_7"  "sd20_8"  "sd20_9"  "sd50_1"  "sd50_2"  "sd50_3"
## [50] "sd50_4"  "sd50_5"  "sd50_6"  "sd50_7"  "sd50_8"  "sd50_9"  "sd100_1"
## [57] "sd100_2" "sd100_3" "sd100_4" "sd100_5" "sd100_6" "sd100_7" "sd100_8"
## [64] "sd100_9"
```

Data Cleaning

The following steps convert the raw Qualtrics export into participant-level indifference points that can be used for modeling.

Reshape Data

This removes rows that do not contain a participant ID. The Social Discounting task contains multiple trials per distance level.
This step converts the dataset from **wide format to long format**, which allows trial-level processing. After this step:

- `trial` contains the trial name (e.g., sd1_1, sd10_5)
- `choice` contains the participant's response.

```
data_wide2 <- data_wide[!is.na(data_wide$subnum), ]

longer_numeric <- pivot_longer(
  data_wide2,
  cols = c(
    "sd1_1","sd1_2","sd1_3","sd1_4","sd1_5","sd1_6","sd1_7","sd1_8","sd1_9",
    "sd2_1","sd2_2","sd2_3","sd2_4","sd2_5","sd2_6","sd2_7","sd2_8","sd2_9",
    "sd5_1","sd5_2","sd5_3","sd5_4","sd5_5","sd5_6","sd5_7","sd5_8","sd5_9",
    "sd10_1","sd10_2","sd10_3","sd10_4","sd10_5","sd10_6","sd10_7","sd10_8","sd10_9",
    "sd20_1","sd20_2","sd20_3","sd20_4","sd20_5","sd20_6","sd20_7","sd20_8","sd20_9",
    "sd50_1","sd50_2","sd50_3","sd50_4","sd50_5","sd50_6","sd50_7","sd50_8","sd50_9",
    "sd100_1","sd100_2","sd100_3","sd100_4","sd100_5","sd100_6","sd100_7","sd100_8","sd1
00_9"
  ),
  names_to = "trial",
  values_to = "choice"
)
```

Split Trial Information

Each trial name contains two pieces of information:

- social distance (e.g., sd100)
- trial number (e.g., 9)

This step separates these into two columns:

- `distance`
- `amount1`

REFERENCE FOR AMOUNTS: This table shows the amount offered for the participant to keep.
The alternative option is always a $75 / $75 split.

- 1 - 155
- 2 - 145
- 3 - 135
- 4 - 125
- 5 - 115
- 6 - 105
- 7 - 95
- 8 - 85
- 9 - 75

```
split_data <- separate(
  longer_numeric,
  trial,
  into = c("distance", "amount1"),
  sep = "_"
)
```

Quality Check

This step flags participants with missing or inconsistent responses at each social distance. Each response is recoded as 0 = kept or 1 = shared (on trial 9, –5 = kept and 0 = shared). For each participant and distance, we then count:

- n_trials: the number of valid responses (should be 9)
- n_switches: the number of times the participant changes between keeping and sharing. 0 = always kept or always shared, 1 = expected pattern, 2 or more = inconsistent.

The output lists any participant × distance with fewer than 9 valid trials or more than one switch. If nothing is listed, the data are complete and consistent. Inspect any flagged rows before continuing. You can keep them, exclude them, or use the logistic regression alternative below. Report the number flagged in your Methods.

```
qc <- split_data %>%
  mutate(amount1 = as.numeric(amount1),
         choice  = as.numeric(choice),
         share = case_when(amount1 == 9 & choice %in% c(-5, 5) ~ 0,  # kept on trial 9
                           amount1 == 9 & choice == 0          ~ 1,  # shared on trial 9
                           amount1 != 9 & choice == 0          ~ 0,  # kept on trials 1-
8
                           amount1 != 9 & choice > 0           ~ 1)) %>% # shared on tri
als 1-8
  arrange(subnum, distance, amount1) %>%
  group_by(subnum, distance) %>%
  summarise(n_trials   = sum(!is.na(share)),
            n_switches = sum(diff(share[!is.na(share)]) != 0),
            .groups = "drop")

qc %>% filter(n_trials < 9 | n_switches > 1)
```

```
## # A tibble: 31 × 4
##    subnum distance n_trials n_switches
##     <int> <chr>       <int>      <int>
##  1     28 sd20            9          3
##  2     33 sd100           9          2
##  3     35 sd100           9          3
##  4     35 sd20            9          2
##  5     42 sd50            9          3
##  6     43 sd20            9          3
##  7     44 sd5             9          2
##  8     47 sd1             9          2
##  9     48 sd20            9          3
## 10     55 sd1             9          3
## # ℹ 21 more rows
```

Recode Choice Values

Due to how the task is coded in Qualtrics:

For most trials:

- 0 = participant kept the money

On the final trial:

- -5 = participant kept the money
- 0 = participant shared

To correctly identify the crossover point, we temporarily convert:

-5 → 5

This ensures that if a participant keeps money on all trials, the final trial is selected as their maximum amount willing to forgo (we select the most positive number).

Later, this value is converted back to -5 to distinguish participants who kept money on all trials whose maximum amount willing to forgo will be 0.

```
split_data$choice <- ifelse(split_data$choice == -5, 5, split_data$choice)
```

Remove Invalid/Missing Responses

This step:

- removes missing responses
- ensures `choice` values are numeric

Then we select rows where choice is above or equal to 0 (why we changed -5 to 5) to highlight valid responses

```
filtered_data <- split_data[!is.na(as.numeric(split_data$choice)), ]
filtered_data <- subset(filtered_data, choice == 0 | choice > 0)
```

Convert Social Distance to Numeric

The distance labels (sd1, sd2, etc.) are converted to numeric values (1, 2, etc), creating a new column called “Distance”

```
max_ratings1 <- filtered_data %>%
  dplyr::select(-any_of("X")) %>%
  mutate(
    Distance = case_when(
      distance == "sd1" ~ 1,
      distance == "sd2" ~ 2,
      distance == "sd5" ~ 5,
      distance == "sd10" ~ 10,
      distance == "sd20" ~ 20,
      distance == "sd50" ~ 50,
      distance == "sd100" ~ 100,
      TRUE ~ NA_real_
    )
  )
```

Compute Indifference Points

We calculate the **maximum amount participants were willing to forgo** for each social distance by identifying the crossover point where participants switch from keeping money to sharing (indifference point).

First, we group decisions by Distance and select the **maximum choice value** at each social distance (the most positive number). This selects the **first** time the participant crosses over from keep to share. This maximum rating/willing to forgo is put in a new column called max_rating. Then we left_join all of the other predictor variables (if you have any other covariates)

Then, in the max_rating column, we convert 5 (kept at the last trial) back to -5.

```
trial_filtered_data <- max_ratings1 %>%
  group_by(subnum, Distance, distance) %>%
  summarise(max_rating = max(choice), .groups = "drop") %>%
  left_join(
    max_ratings1 %>%
      distinct(subnum, .keep_all = TRUE) %>%
      dplyr::select(-Distance, -distance, -amount1, -choice),
    by = "subnum"
  )
trial_filtered_data$max_rating <- ifelse(trial_filtered_data$max_rating == 5, -5, trial_
filtered_data$max_rating)
```

Data Check

This table shows the distribution of indifference points across participants.

These values reflect the maximum amount participants were willing to forgo at each social distance.

Interpretation:

- 80 = shared on the first trial
- 70–10 = switched to sharing on trials 2-8 (later crossover = less generous)
- 0 = shared on the final trial
- -5 = kept money on all trials

```
table(trial_filtered_data$max_rating)
```

```
##
##  -5   0  10  20  30  40  50  60  70  80
##  43 147 111  79  55  61  82  58 104 100
```

OPTIONAL: Missing Data - Data Cleaning Alternative

The standard social discounting pipeline identifies indifference points by locating the crossover trial where participants switch from keeping money to sharing. This requires complete data within each social distance. However, in some context (neuroimaging studies, online data collection without request response), participants may have a lot of missing data on individual trials and you may not want to pairwise delete. To address this, we use a trial-level logistic regression approach, which uses all available trials to model choices directly and estimate the indifference point.

Reshape Data and Split Trial Information

```
data_wide2 <- data_wide[!is.na(data_wide$subnum), ]

longer_numeric <- pivot_longer(
  data_wide2,
  cols = c(
    "sd1_1","sd1_2","sd1_3","sd1_4","sd1_5","sd1_6","sd1_7","sd1_8","sd1_9",
    "sd2_1","sd2_2","sd2_3","sd2_4","sd2_5","sd2_6","sd2_7","sd2_8","sd2_9",
    "sd5_1","sd5_2","sd5_3","sd5_4","sd5_5","sd5_6","sd5_7","sd5_8","sd5_9",
    "sd10_1","sd10_2","sd10_3","sd10_4","sd10_5","sd10_6","sd10_7","sd10_8","sd10_9",
    "sd20_1","sd20_2","sd20_3","sd20_4","sd20_5","sd20_6","sd20_7","sd20_8","sd20_9",
    "sd50_1","sd50_2","sd50_3","sd50_4","sd50_5","sd50_6","sd50_7","sd50_8","sd50_9",
    "sd100_1","sd100_2","sd100_3","sd100_4","sd100_5","sd100_6","sd100_7","sd100_8","sd1
00_9"
  ),
  names_to = "trial",
  values_to = "choice"
)

split_data <- separate(
  longer_numeric,
  trial,
  into = c("distance", "amount1"),
  sep = "_"
)
```

Recode Choice Values

Here we recode the amount1 column to be amount_value which reflects the amount of money forgone when choosing to share, such that higher values correspond to greater amount forgone. We also recode the choice column to be binary where 0 is keep and 1 is share.

```
split_data$choice <- as.numeric(split_data$choice)

logit_data <- split_data %>%
  mutate(
    amount1 = as.numeric(amount1),

    # Convert trial number to actual dollar values
    amount_value = case_when(
      amount1 == 1 ~ 80,
      amount1 == 2 ~ 70,
      amount1 == 3 ~ 60,
      amount1 == 4 ~ 50,
      amount1 == 5 ~ 40,
      amount1 == 6 ~ 30,
      amount1 == 7 ~ 20,
      amount1 == 8 ~ 10,
      amount1 == 9 ~ 0,
      TRUE ~ NA_real_
    ),

    Distance = case_when(
      distance == "sd1" ~ 1,
      distance == "sd2" ~ 2,
      distance == "sd5" ~ 5,
      distance == "sd10" ~ 10,
      distance == "sd20" ~ 20,
      distance == "sd50" ~ 50,
      distance == "sd100" ~ 100,
      TRUE ~ NA_real_
    ),

    # Recode choice: 0 = keep, 1 = share
    choice_binary = case_when(
      amount1 == 9 & choice == -5 ~ 0,
      amount1 == 9 & choice == 0 ~ 1,

      amount1 != 9 & choice == 0 ~ 0,
      amount1 != 9 & choice > 0 ~ 1
    )
  )
```

Estimate Indifference Point

Here we estimate indifference points using a trial-level logistic regression model for each participant at each social distance. This approach uses all available observed trials rather than relying on a single crossover point.

Participants with no variability in responses (i.e., always choosing to keep or always choosing to share) are assigned boundary values of the task range, as logistic regression cannot be reliably estimated in the absence of response variability. Additionally, the indifference point for participants with fewer than 3 responses for a social distance are not estimated. We model the probability of choosing to share (choice_binary = 1) as a function of the amount of money forgone (amount_value). The resulting function describes how decision probability changes as a

function of increasing monetary cost. The indifference point is defined as the value of amount_value at which the participant is equally likely to keep or share the reward (i.e., P(share) = 0.5). This is computed analytically from the fitted logistic regression coefficients.

```
indifference_data <- logit_data %>%
  group_by(subnum, Distance) %>%
  group_modify(~{

    df <- .x

    # Remove missing rows (handled automatically, but explicit here)
    df <- df %>% filter(!is.na(choice_binary), !is.na(amount_value))

    # Handle edge cases
    if (nrow(df) < 3) {
      return(tibble(max_rating = NA_real_))
    }

    if (all(df$choice_binary == 1)) {
      return(tibble(max_rating = 80))  # always shared
    }

    if (all(df$choice_binary == 0)) {
      return(tibble(max_rating = 0))    # always kept
    }

    # Fit logistic regression
    fit <- glm(choice_binary ~ amount_value,
               family = binomial,
               data = df)

    coefs <- coef(fit)

    # Compute indifference point (P = 0.5)
    v <- -coefs[1] / coefs[2]

    tibble(max_rating = v)
  }) %>%
  ungroup()

trial_filtered_data <- indifference_data %>%
  left_join(
    logit_data %>%
      distinct(subnum, .keep_all = TRUE) %>%
      dplyr::select(-Distance, -distance, -amount1, -amount_value, -choice_binary, -choi
ce),
    by = "subnum"
  )

summary(trial_filtered_data$max_rating)
```

Analysis

This code uses the cleaned data to run the hyperbolic model, extract individual logk values and calculate individual AUC values

Hyperbolic Model

The Social Discounting Task is typically analyzed using a hyperbolic discounting model, which estimates how generosity decreases as social distance increases.

The model estimates two parameters:

- v0: baseline generosity (at social distance = 0)
- logk: social discounting rate

Higher (less negative) logk values indicate steeper discounting across social distance.

Model 1: Base model (no predictors)

This model estimates the hyperbolic social discounting function without including any predictors. It is used to obtain **participant-level discounting parameters**, including logk, which reflects the rate of social discounting. In most cases, researchers should run this model first and extract the **individual logk estimates** from it. These logk values can then be used in subsequent analyses (e.g., regression models) that include covariates or group variables. Most users should run this model.

Starting values are chosen automatically with `fit_best_start()` (see Model Fitting Functions). `start_grid` lists the starting values to try for v0 and logk. Widen it if no starting values converge.

```
# Starting values to try (long form: v0 between 0 and 80)
start_grid <- expand.grid(v0 = c(40, 60, 80), logk = c(-6, -4, -2, -1, 0))

hyp_fit <- function(st) nlme(as.numeric(max_rating) ~ v0 / (1 + exp(logk) * Distance),
                             fixed     = list(v0 ~ 1, logk ~ 1),
                             random    = v0 + logk ~ 1 | subnum,
                             start     = st,
                             data      = trial_filtered_data,
                             control   = ctl,
                             na.action = na.omit)

base_model_long <- fit_best_start(hyp_fit, start_grid)
summary(base_model_long)
```

```
## Nonlinear mixed-effects model fit by maximum likelihood
##   Model: as.numeric(max_rating) ~ v0/(1 + exp(logk) * Distance)
##   Data: trial_filtered_data
##        AIC      BIC    logLik
##   5779.216 5807.616 -2883.608
##
## Random effects:
##  Formula: list(v0 ~ 1, logk ~ 1)
##  Level: subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##          StdDev   Corr
## v0       2.327961 v0
## logk     1.927658 1
## Residual 5.309034
##
## Fixed effects:  list(v0 ~ 1, logk ~ 1)
##         Value Std.Error  DF   t-value p-value
## v0   84.34615 0.5352534 719 157.58172       0
## logk -1.77646 0.1777845 719  -9.99223       0
##  Correlation:
##      v0
## logk 0.474
##
## Standardized Within-Group Residuals:
##        Min         Q1        Med         Q3        Max
## -3.2097775 -0.8186727 -0.1632597  0.4665593  3.3610735
##
## Number of Observations: 840
## Number of Groups: 120
```

```
attr(base_model_long, "start_search")   # every starting value tried and the result it r
eached
```

```
##    start_v0 start_logk    logLik   est_v0  est_logk
## 1        40         -6 -2888.037 81.85256 -1.825930
## 2        60         -6 -2888.038 81.85262 -1.825942
## 3        80         -6 -2888.037 81.85227 -1.825962
## 4        40         -4 -2883.608 84.34615 -1.776463
## 5        60         -4 -2888.050 81.85307 -1.826037
## 6        80         -4 -2888.049 81.85308 -1.826033
## 7        40         -2 -2883.616 84.34318 -1.776515
## 8        80         -2 -2883.621 84.34312 -1.776553
## 9        60         -1 -2888.053 81.85322 -1.826058
## 10       80         -1 -2883.610 84.34527 -1.776473
## 11       40          0 -2888.058 81.85381 -1.826064
```

Comparison Models

We compare the hyperbolic model with an exponential and a linear model to confirm that the hyperbolic model fits best.

- **Exponential model:** like the hyperbolic model, it is parameterized with logk (k = exp(logk)), which keeps k positive. Its starting values are found with `fit_best_start()`. Note that the exponential logk is not on the same scale as the hyperbolic logk, so the two should not be compared directly.
- **Linear model:** this model is linear, so it is fit with `lme()`, which does not need starting values. `method = "ML"` is required so that its AICc can be compared with the `nlme` models.

```
# Exponential model
exp_fit <- function(st) nlme(as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Distance),
                             fixed     = list(v0 ~ 1, logk ~ 1),
                             random    = v0 + logk ~ 1 | subnum,
                             start     = st,
                             data      = trial_filtered_data,
                             control   = ctl,
                             na.action = na.omit)

exponential_model_base <- fit_best_start(exp_fit, start_grid)
summary(exponential_model_base)
```

```
## Nonlinear mixed-effects model fit by maximum likelihood
##   Model: as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Distance)
##   Data: trial_filtered_data
##        AIC      BIC    logLik
##   5909.184 5937.584 -2948.592
##
## Random effects:
##  Formula: list(v0 ~ 1, logk ~ 1)
##  Level: subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##          StdDev   Corr
## v0       4.910341 v0
## logk     1.617547 -1
## Residual 5.716229
##
## Fixed effects:  list(v0 ~ 1, logk ~ 1)
##         Value Std.Error  DF   t-value p-value
## v0   72.36110 0.6218219 719 116.36949       0
## logk -2.57824 0.1489490 719 -17.30954       0
##  Correlation:
##      v0
## logk -0.667
##
## Standardized Within-Group Residuals:
##           Min            Q1           Med            Q3           Max
## -2.501169e+00 -5.383591e-01 -4.034352e-05  6.209788e-01  3.342495e+00
##
## Number of Observations: 840
## Number of Groups: 120
```

```
# Linear model
linear_model_base <- lme(as.numeric(max_rating) ~ Distance,
                         random    = ~ Distance | subnum,
                         data      = trial_filtered_data,
                         method    = "ML",
                         na.action = na.omit,
                         control   = lmeControl(opt = "optim"))
summary(linear_model_base)
```

```
## Linear mixed-effects model fit by maximum likelihood
##   Data: trial_filtered_data
##        AIC      BIC    logLik
##   7211.766 7240.167 -3599.883
##
## Random effects:
##  Formula: ~Distance | subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##             StdDev     Corr
## (Intercept) 22.3966776 (Intr)
## Distance     0.1042712 -0.796
## Residual    14.2910597
##
## Fixed effects:  as.numeric(max_rating) ~ Distance
##                Value Std.Error  DF   t-value p-value
## (Intercept) 47.18096 2.1419558 719  22.02705       0
## Distance    -0.45244 0.0174523 719 -25.92424       0
##  Correlation:
##          (Intr)
## Distance -0.569
##
## Standardized Within-Group Residuals:
##        Min         Q1        Med         Q3        Max
## -2.0784481 -0.7125910 -0.0437131  0.6341220  2.8268318
##
## Number of Observations: 840
## Number of Groups: 120
```

Model Comparison (AICc Weights)

After fitting the hyperbolic, exponential, and linear models, we compare their relative fit using Akaike's Information Criterion corrected for sample size (AICc). AICc balances model fit and complexity: lower values indicate better-fitting models, with a penalty for additional parameters.

The `aic_table()` function (see Model Fitting Functions) returns one row per model, ranked from best to worst fit:

- K: number of estimated parameters
- LL: log-likelihood
- AICc
- Delta: ΔAICc, the difference between each model's AICc and the lowest AICc among the models
- Weight: AICc weight, the relative likelihood that each model is the best of the candidate models, calculated as

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum_j \exp(-\Delta_j/2)}$$

- Cum_Weight: cumulative weight

We also calculate AIC weights directly from each model's AIC, without the small-sample correction:

- Extracts AIC for each model using `AIC()`
- Computes ΔAIC as the difference from the lowest AIC
- Calculates AIC weights using the same formula as above

With large samples (840 observations in the example data), AIC and AICc are nearly identical, so both tables lead to the same conclusions.

Interpretation:

In most cases, the hyperbolic model will have the lowest AICc, so the exponential and linear models will have positive ΔAICc values. ΔAICc < 2 → substantial support (models are essentially equivalent) ΔAICc 4–7 → considerably less support ΔAICc > 10 → essentially no support (the model with the lower AICc fits better)

AICc weights sum to 1 across the models. An AICc weight of 1 for the hyperbolic model means it is overwhelmingly the best-fitting model compared to the exponential and linear alternatives.

```
models_base <- list(hyperbolic  = base_model_long,
                    exponential = exponential_model_base,
                    linear      = linear_model_base)

AICc_table_base <- aic_table(models_base)
print(AICc_table_base)
```

```
##         Model K        LL   n     AICc     Delta        Weight Cum_Weight
## 1  hyperbolic 6 -2883.608 840 5779.317    0.0000  1.000000e+00          1
## 2 exponential 6 -2948.592 840 5909.284  129.9677  5.996086e-29          1
## 3      linear 6 -3599.883 840 7211.867 1432.5505 8.425855e-312          1
```

```
# AIC weights (manual)
AICs_base <- sapply(models_base, AIC)
delta_base <- AICs_base - min(AICs_base)
weights_base <- exp(-0.5 * delta_base) / sum(exp(-0.5 * delta_base))

AIC_table_base <- data.frame(
  Model = names(AICs_base),
  AIC = as.numeric(AICs_base),
  Delta = delta_base,
  Weight = weights_base
)
print(AIC_table_base)
```

```
##                   Model      AIC     Delta        Weight
## hyperbolic   hyperbolic 5779.216    0.0000  1.000000e+00
## exponential exponential 5909.184  129.9677  5.996086e-29
## linear           linear 7211.766 1432.5505 8.425855e-312
```

Extract logk

Below we extract individual logk values for each participant. In most cases, logk values should be extracted from the base model (Model 1). This ensures that individual discounting estimates are derived directly from task behavior and are not influenced by predictors included in the hyperbolic model. These participant-level logk values can then be used in subsequent analyses (e.g., regression models with covariates).

```
cf <- coef(base_model_long)                  # fixed + random, one row per subnum
individual_logk_df_base <- data.frame(
  subnum    = type.convert(rownames(cf), as.is = TRUE),
  logk_base = cf$logk
)
head(individual_logk_df_base)
```

```
##   subnum  logk_base
## 1      1 -0.8300607
## 2      2 -4.0841223
## 3      3 -0.6013120
## 4      4  2.1649239
## 5      5 -4.8288061
## 6      6 -0.1228062
```

Check Against the True Values (Practice Data Only)

The practice data were simulated from known parameters, saved in `SDlongform_exampledata_trueparameters.csv`. This chunk compares the estimated logk with the true logk. It runs only if that file is in the `path` folder, so it is skipped for real data.

```
true_file_long <- file.path(path, sub("\\.csv$", "_trueparameters.csv", sd_long))

if (file.exists(true_file_long)) {
  true_long <- read.csv(true_file_long)

  check_long <- individual_logk_df_base %>%
    inner_join(true_long, by = "subnum")

  print(cor.test(check_long$logk, check_long$logk_base)) # correlation between true and
estimated logk
}
```

```
##
##  Pearson's product-moment correlation
##
## data:  check_long$logk and check_long$logk_base
## t = 108.37, df = 118, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.9928437 0.9965269
## sample estimates:
##       cor
## 0.9950137
```

Extracting AUC

Area under the curve (AUC) is computed using the trapezoidal method (trapz) across the observed social distance values (1, 2, 5, 10, 20, 50, 100). Because AUC is calculated over the actual range of x-values included in the task, the total span of the social distance axis is 100 − 1 = 99 units. The maximum possible AUC therefore corresponds to a participant consistently selecting the maximum forgone amount (80) across this range, yielding a value of 99 × 80 = 7,920. Observed AUC values are divided by 7,920 to yield a normalized index bounded between 0 and 1.

```
trial_filtered_data2 <- dplyr::left_join(
  trial_filtered_data,
  individual_logk_df_base,
  by = "subnum"
)

AUC_long <- trial_filtered_data2 %>%
  group_by(subnum) %>%
  summarise(AUC = trapz(as.numeric(Distance), as.numeric(max_rating))/7920, .groups = "d
rop") %>%
  left_join(
    trial_filtered_data2 %>%
      dplyr::distinct(subnum, .keep_all = TRUE) %>%
      dplyr::select(-max_rating, -Distance, -any_of(c("distance","amount1","choice","tri
al"))),
    by = "subnum"
  )

cor.test(AUC_long$logk_base, AUC_long$AUC)
```

```
##
##  Pearson's product-moment correlation
##
## data:  AUC_long$logk_base and AUC_long$AUC
## t = -29.808, df = 118, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.9575401 -0.9142863
## sample estimates:
##        cor
## -0.9395553
```

```
mean(AUC_long$AUC)
```

```
## [1] 0.2455755
```

```
sd(AUC_long$AUC)
```

```
## [1] 0.2597602
```

```
range(AUC_long$AUC)
```

```
## [1] -0.04261364  0.93686869
```

Model 2: Model With Predictors (Optional)

You may want to examine how participant characteristics predict differences in the hyperbolic discounting function itself. This model allows predictors (e.g., demographic variables, personality traits, or group membership) to explain variability in the discounting rate (logk). This step is **optional** and is typically used only if the goal is to test predictors **directly within the hyperbolic model**, rather than analyzing extracted logk values in a separate model.

Tips for using this model:

Modify the predictors in the example to match the variables available in your dataset. Set `n_extra` in `fit_best_start()` to the number of predictors in the model (5 in this example). The starting values for the predictor slopes are set to 0.

```
pred_hyp_fit <- function(st) nlme(as.numeric(max_rating) ~ v0 / (1 + exp(logk) * Distanc
e),
                                  fixed     = list(v0 ~ 1,
                                                   logk ~ as.numeric(Male) + as.numeric
(age) + as.numeric(income) +
                                                          as.numeric(FIT_sum) + as.numer
ic(Group_var)),
                                  random    = v0 + logk ~ 1 | subnum,
                                  start     = st,
                                  data      = trial_filtered_data,
                                  control   = ctl,
                                  na.action = na.omit)

predictor_model <- fit_best_start(pred_hyp_fit, start_grid, n_extra = 5)
summary(predictor_model)
attr(predictor_model, "start_search")
```

Comparison Models

We compare the hyperbolic model with an exponential and a linear model to confirm that the hyperbolic model fits best.

- **Exponential model:** like the hyperbolic model, it is parameterized with logk (k = exp(logk)), which keeps k positive. Its starting values are found with `fit_best_start()`. Note that the exponential logk is not on the same scale as the hyperbolic logk, so the two should not be compared directly.
- **Linear model:** this model is linear, so it is fit with `lme()`, which does not need starting values. `method = "ML"` is required so that its AICc can be compared with the `nlme` models. In `lme()`, a predictor's effect on the slope is written as an interaction with Distance (`Distance:predictor`), so the predictors affect the rate of decline, as in the hyperbolic and exponential models.

```
# Exponential model
pred_exp_fit <- function(st) nlme(as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Distanc
e),
                                  fixed     = list(v0 ~ 1,
                                                   logk ~ as.numeric(Male) + as.numeric
(age) + as.numeric(income) +
                                                          as.numeric(FIT_sum) + as.numer
ic(Group_var)),
                                  random    = v0 + logk ~ 1 | subnum,
                                  start     = st,
                                  data      = trial_filtered_data,
                                  control   = ctl,
                                  na.action = na.omit)

exponential_model_pred <- fit_best_start(pred_exp_fit, start_grid, n_extra = 5)
summary(exponential_model_pred)

# Linear model
linear_model_pred <- lme(as.numeric(max_rating) ~ Distance +
                           Distance:(as.numeric(Male) + as.numeric(age) + as.numeric(inc
ome) +
                                       as.numeric(FIT_sum) + as.numeric(Group_var)),
                         random    = ~ Distance | subnum,
                         data      = trial_filtered_data,
                         method    = "ML",
                         na.action = na.omit,
                         control   = lmeControl(opt = "optim"))
summary(linear_model_pred)
```

Model Comparison (AICc Weights)

After fitting the hyperbolic, exponential, and linear models, we compare their relative fit using Akaike's Information Criterion corrected for sample size (AICc). AICc balances model fit and complexity: lower values indicate better-fitting models, with a penalty for additional parameters.

The `aic_table()` function (see Model Fitting Functions) returns one row per model, ranked from best to worst fit:

- K: number of estimated parameters
- LL: log-likelihood
- AICc
- Delta: ΔAICc, the difference between each model's AICc and the lowest AICc among the models
- Weight: AICc weight, the relative likelihood that each model is the best of the candidate models, calculated as

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum_j \exp(-\Delta_j/2)}$$

- Cum_Weight: cumulative weight

We also calculate AIC weights directly from each model's AIC, without the small-sample correction:

- Extracts AIC for each model using `AIC()`
- Computes ΔAIC as the difference from the lowest AIC
- Calculates AIC weights using the same formula as above

With large samples (840 observations in the example data), AIC and AICc are nearly identical, so both tables lead to the same conclusions.

Interpretation:

In most cases, the hyperbolic model will have the lowest AICc, so the exponential and linear models will have positive ΔAICc values. ΔAICc < 2 → substantial support (models are essentially equivalent) ΔAICc 4–7 → considerably less support ΔAICc > 10 → essentially no support (the model with the lower AICc fits better)

AICc weights sum to 1 across the models. An AICc weight of 1 for the hyperbolic model means it is overwhelmingly the best-fitting model compared to the exponential and linear alternatives.

```
models_pred <- list(hyperbolic  = predictor_model,
                    exponential = exponential_model_pred,
                    linear      = linear_model_pred)

# AICc table (same values as aictab() in the AICcmodavg package)
AICc_table_pred <- aic_table(models_pred)
AICc_table_pred

# AIC weights (manual)
AICs_pred <- sapply(models_pred, AIC)
delta_pred <- AICs_pred - min(AICs_pred)
weights_pred <- exp(-0.5 * delta_pred) / sum(exp(-0.5 * delta_pred))
weights_pred

AIC_table_pred <- data.frame(
  Model = names(AICs_pred),
  AIC = as.numeric(AICs_pred),
  Delta = delta_pred,
  Weight = weights_pred
)
AIC_table_pred
```

Extracting logk values

In the predictor model, each participant's logk depends on their predictor values. It is calculated as the participant-specific intercept (the fixed intercept plus that participant's random effect) plus each fixed slope multiplied by the participant's value on that predictor. The code below:

- uses coef() to get each participant's model coefficients. The logk intercept already includes the participant's random effect.

- selects one row per participant from the data and puts the rows in the same order as the model output. Participants the model excluded (e.g., because of missing covariates) are dropped.
- creates a predictor matrix with each participant's covariate values. The predictors must match those in the hyperbolic model, in the same order.
- multiplies each participant's predictor values by the corresponding coefficients and sums them to get that participant's logk
- creates a dataset (individual_logk_df_pred) with one row per participant, containing the participant ID and estimated logk.

```
cf  <- coef(predictor_model)
pp  <- trial_filtered_data %>% distinct(subnum, .keep_all = TRUE)
pp  <- pp[match(type.convert(rownames(cf), as.is = TRUE), pp$subnum), ] #puts the rows i
n the same order as the participants in cf. also drops anyone that the model left out (e
x: for missing a covariate)
X <- model.matrix(~ as.numeric(Male) + as.numeric(age) + as.numeric(income) +
                    as.numeric(FIT_sum) + as.numeric(Group_var),
                  data = pp) #builds a matrix

individual_logk_df_pred <- data.frame(
  subnum    = pp$subnum,
  logk_pred = rowSums(X * as.matrix(cf[, grep("^logk", names(cf))]))
)
```

Extracting AUC

```
if (exists("AUC_long")) {
  AUC_long <- dplyr::left_join(
    AUC_long,
    individual_logk_df_pred,
    by = "subnum"
  )
} else {
  trial_filtered_data3 <- dplyr::left_join(
    trial_filtered_data,
    individual_logk_df_pred,
    by = "subnum"
  )

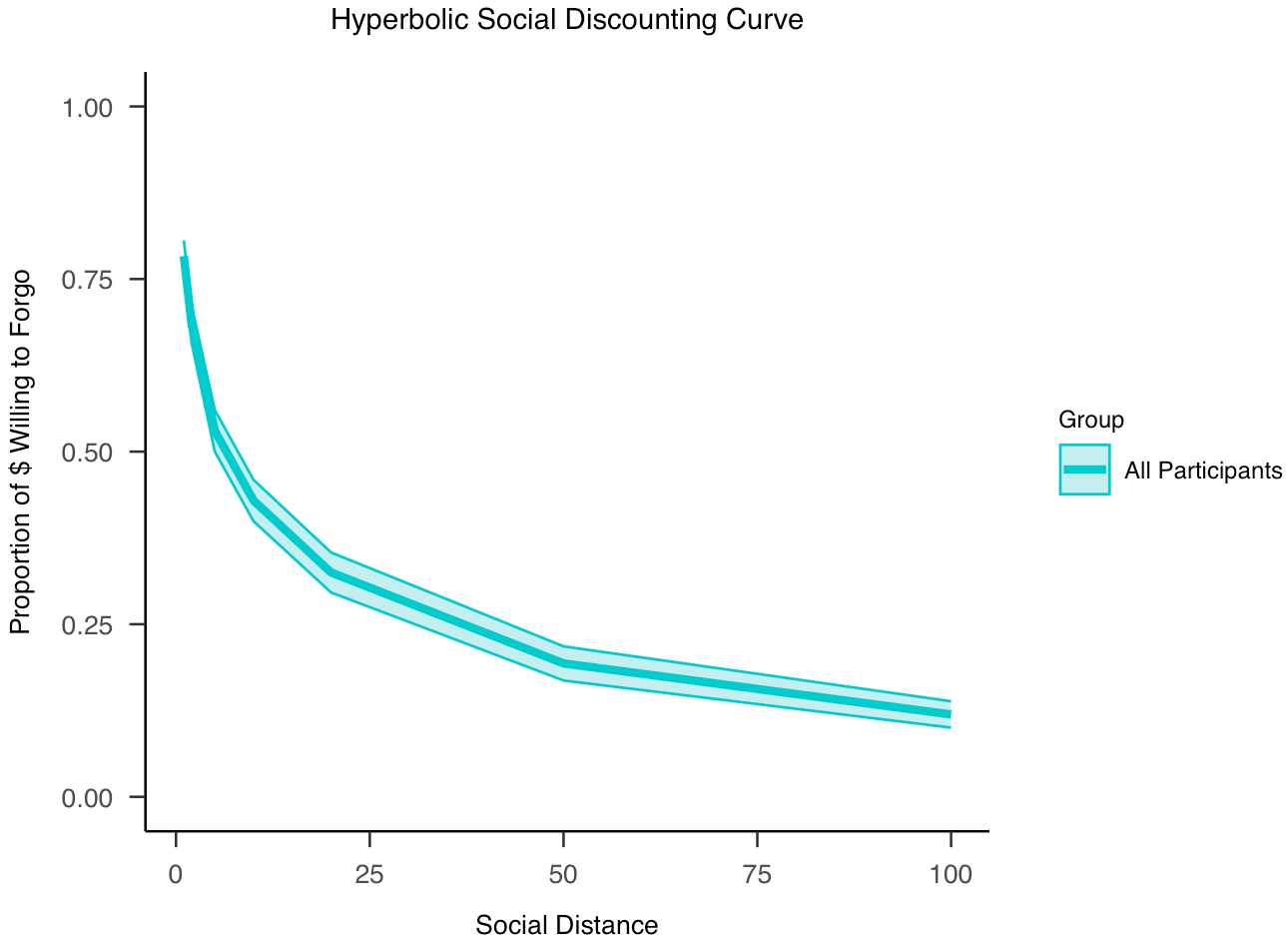
  AUC_long <- trial_filtered_data3 %>%
    group_by(subnum) %>%
    summarise(AUC = trapz(as.numeric(Distance), as.numeric(max_rating))/7920, .groups =
"drop") %>%
    left_join(
      trial_filtered_data3 %>%
        dplyr::distinct(subnum, .keep_all = TRUE) %>%
        dplyr::select(-max_rating, -Distance, -any_of(c("distance","amount1","choice","t
rial"))),
      by = "subnum"
    )
}
```

Graph of Social Discounting curve

This code generates a plot of participants' generosity across social distance, scaled as a proportion of the maximum possible amount (percentage). The workflow automatically adapts based on whether the dataset contains a grouping variable (Group):

- With "Group_var" present: The data are summarized separately for each group. The plot displays one line per group with shaded ribbons representing ±1 standard error of the mean. Groups are automatically labeled "Group1" and "Group2," and color-coded for clarity.
- Without "Group_var": The data are summarized across all participants, producing a single hyperbolic curve with its standard error ribbon.

This approach ensures the graph works whether or not multiple groups are present, providing a clear visualization of social discounting behavior in your sample.



SD SHORT FORM

Import Data

This section imports the Qualtrics data file.

```
data_wide_short <- read.delim(
  file = paste0(path, sd_short),
  sep = ",",
  quote = "\"",
  header = TRUE
)

sd_cols <- grep("X*_SDshort_decision", names(data_wide_short), value = TRUE)

data_wide_short <- data_wide_short %>%
  mutate(across(all_of(sd_cols), ~ suppressWarnings(as.numeric(.x)))) %>%
  filter(if_any(all_of(sd_cols), ~ !is.na(.x))) %>%   # drops the 2 Qualtrics header row
s
  mutate(subnum = type.convert(na_if(trimws(as.character(subnum)), ""), as.is = TRUE))
# numeric IDs stay numeric (done after the header rows are removed)
```

- 1_SDshort_decision = N1
- 2_SDshort_decision = N2
- 3_SDshort_decision = N5
- 4_SDshort_decision = N10
- 5_SDshort_decision = N20
- 6_SDshort_decision = N50
- 7_SDshort_decision = N100

Data Cleaning

The following steps convert the raw Qualtrics export into participant-level indifference points that can be used for modeling.

Reshape Data

This removes rows that do not contain a participant ID. The Social Discounting task short form contains one trial per distance level.
This step converts the dataset from **wide format to long format**, which allows trial-level processing. After this step:

- `trial` contains the trial name (e.g., X1_SDshort_decision, X2_SDshort_decision)
- `max_rating` contains the participant's response.

```
data_wide_short2 <- data_wide_short[!is.na(data_wide_short$subnum), ]

longer_numeric_short <- pivot_longer(
  data_wide_short2,                          # pivot_longer keeps all other columns
  cols = matches("^X[1-7]_SDshort_decision$"),
  names_to = "trial",
  values_to = "max_rating"
)
```

```
longer_numeric_short$distance <- sub("_SDshort_decision", "", longer_numeric_short$tria
l)
```

Remove Invalid/Missing Responses

This step:

- removes missing responses
- ensures `max_rating` values are numeric

```
filtered_data_short <- longer_numeric_short[!is.na(as.numeric(longer_numeric_short$max_r
ating)), ]
filtered_data_short <- subset(filtered_data_short, max_rating >= 0 & max_rating <= 10)
```

```
trial_filtered_data_s <- filtered_data_short %>%
  dplyr::select(-any_of("X")) %>%
  mutate(
    Distance = case_when(
      distance == "X1" ~ 1,
      distance == "X2" ~ 2,
      distance == "X3" ~ 5,
      distance == "X4" ~ 10,
      distance == "X5" ~ 20,
      distance == "X6" ~ 50,
      distance == "X7" ~ 100,
      TRUE ~ NA_real_
    )
  )

table(trial_filtered_data_s$max_rating)
```

```
##
##   0   1   2   3   4   5   6   7   8   9  10
##  59 119 114 132 134 110  91  47  22   7   5
```

Analysis

This code uses the cleaned data to run the hyperbolic model, extract individual logk values and calculate individual AUC values

Hyperbolic Model

The Social Discounting Task is typically analyzed using a hyperbolic discounting model, which estimates how generosity decreases as social distance increases. The model estimates two parameters:

- v0: baseline generosity (at social distance = 0)
- logk: social discounting rate

Higher (less negative) logk values indicate steeper discounting across social distance.

Model 1: Base model (no predictors)

This model estimates the hyperbolic social discounting function without including any predictors. It is used to obtain **participant-level discounting parameters**, including logk, which reflects the rate of social discounting.

In most cases, researchers should run this model first and extract the **individual logk estimates** from it. These logk values can then be used in subsequent analyses (e.g., regression models) that include covariates or group variables. Most users should run this model.

Starting values are chosen automatically with `fit_best_start()` (see Model Fitting Functions). `start_grid` lists the starting values to try for v0 and logk. Widen it if no starting values converge.

```
# Starting values to try (short form: v0 between 0 and 10)
start_grid_s <- expand.grid(v0 = c(4, 7, 10), logk = c(-6, -4, -2, -1, 0))

hyp_fit_s <- function(st) nlme(as.numeric(max_rating) ~ v0 / (1 + exp(logk) * Distance),
                               fixed     = list(v0 ~ 1, logk ~ 1),
                               random    = v0 + logk ~ 1 | subnum,
                               start     = st,
                               data      = trial_filtered_data_s,
                               control   = ctl,
                               na.action = na.omit)

base_model_short <- fit_best_start(hyp_fit_s, start_grid_s)
summary(base_model_short)
```

```
## Nonlinear mixed-effects model fit by maximum likelihood
##   Model: as.numeric(max_rating) ~ v0/(1 + exp(logk) * Distance)
##   Data: trial_filtered_data_s
##        AIC      BIC   logLik
##   2186.161 2214.561 -1087.08
##
## Random effects:
##  Formula: list(v0 ~ 1, logk ~ 1)
##  Level: subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##          StdDev    Corr
## v0       2.1062426 v0
## logk     1.5973798 0.382
## Residual 0.5534536
##
## Fixed effects:  list(v0 ~ 1, logk ~ 1)
##          Value Std.Error  DF   t-value p-value
## v0    5.569961 0.2016811 719  27.61766       0
## logk -3.521554 0.1534130 719 -22.95473       0
##  Correlation:
##      v0
## logk 0.397
##
## Standardized Within-Group Residuals:
##         Min          Q1         Med          Q3         Max
## -2.82846625 -0.61216069 -0.05367477  0.56301605  2.60391277
##
## Number of Observations: 840
## Number of Groups: 120
```

```
attr(base_model_short, "start_search")   # every starting value tried and the result it reached
```

```
##    start_v0 start_logk    logLik   est_v0  est_logk
## 1         4         -6 -1087.081 5.569850 -3.521641
## 2         7         -6 -1087.091 5.569936 -3.521281
## 3        10         -6 -1087.090 5.569927 -3.521296
## 4         4         -4 -1087.091 5.569378 -3.521478
## 5         7         -4 -1087.080 5.569864 -3.521636
## 6        10         -4 -1087.086 5.569400 -3.521568
## 7         4         -2 -1087.091 5.569938 -3.521292
## 8         7         -2 -1087.091 5.569938 -3.521285
## 9        10         -2 -1087.091 5.569935 -3.521274
## 10        4         -1 -1087.084 5.569767 -3.521495
## 11        7         -1 -1087.091 5.569944 -3.521283
## 12       10         -1 -1087.080 5.569961 -3.521554
## 13        4          0 -1439.964 6.326961 -2.648091
## 14        7          0 -1439.975 6.327398 -2.647704
```

Comparison Models

We compare the hyperbolic model with an exponential and a linear model to confirm that the hyperbolic model fits best.

- **Exponential model:** like the hyperbolic model, it is parameterized with logk (k = exp(logk)), which keeps k positive. Its starting values are found with `fit_best_start()`. Note that the exponential logk is not on the same scale as the hyperbolic logk, so the two should not be compared directly.
- **Linear model:** this model is linear, so it is fit with `lme()`, which does not need starting values. `method = "ML"` is required so that its AICc can be compared with the `nlme` models.

```
# Exponential model
exp_fit_s <- function(st) nlme(as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Distance),
                               fixed     = list(v0 ~ 1, logk ~ 1),
                               random    = v0 + logk ~ 1 | subnum,
                               start     = st,
                               data      = trial_filtered_data_s,
                               control   = ctl,
                               na.action = na.omit)

exponential_model_base <- fit_best_start(exp_fit_s, start_grid_s)
summary(exponential_model_base)
```

```
## Nonlinear mixed-effects model fit by maximum likelihood
##   Model: as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Distance)
##   Data: trial_filtered_data_s
##        AIC      BIC    logLik
##   2308.895 2337.295 -1148.447
##
## Random effects:
##  Formula: list(v0 ~ 1, logk ~ 1)
##  Level: subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##          StdDev    Corr
## v0       1.8861583 v0
## logk     1.2721466 0.144
## Residual 0.5978565
##
## Fixed effects:  list(v0 ~ 1, logk ~ 1)
##          Value Std.Error  DF   t-value p-value
## v0    5.156104 0.1768279 719  29.15889       0
## logk -4.028343 0.1220220 719 -33.01326       0
##  Correlation:
##      v0
## logk 0.167
##
## Standardized Within-Group Residuals:
##         Min          Q1         Med          Q3         Max
## -2.63941009 -0.51664909  0.01008927  0.66887143  3.21474331
##
## Number of Observations: 840
## Number of Groups: 120
```

```
# Linear model
linear_model_base <- lme(as.numeric(max_rating) ~ Distance,
                         random    = ~ Distance | subnum,
                         data      = trial_filtered_data_s,
                         method    = "ML",
                         na.action = na.omit,
                         control   = lmeControl(opt = "optim"))
summary(linear_model_base)
```

```
## Linear mixed-effects model fit by maximum likelihood
##   Data: trial_filtered_data_s
##        AIC      BIC    logLik
##   2843.399 2871.799 -1415.699
##
## Random effects:
##  Formula: ~Distance | subnum
##  Structure: General positive-definite, Log-Cholesky parametrization
##             StdDev    Corr
## (Intercept) 1.7763164 (Intr)
## Distance    0.0144263 -0.64
## Residual    1.0008813
##
## Fixed effects:  as.numeric(max_rating) ~ Distance
##                 Value  Std.Error  DF   t-value p-value
## (Intercept)  4.417699 0.16825170 719  26.25649       0
## Distance    -0.031820 0.00166942 719 -19.06072       0
##  Correlation:
##          (Intr)
## Distance -0.588
##
## Standardized Within-Group Residuals:
##         Min          Q1         Med          Q3         Max
## -2.63719175 -0.53963525  0.01995536  0.50396325  3.50832088
##
## Number of Observations: 840
## Number of Groups: 120
```

Model Comparison (AICc Weights)

After fitting the hyperbolic, exponential, and linear models, we compare their relative fit using Akaike's Information Criterion corrected for sample size (AICc). AICc balances model fit and complexity: lower values indicate better-fitting models, with a penalty for additional parameters.

The `aic_table()` function (see Model Fitting Functions) returns one row per model, ranked from best to worst fit:

- K: number of estimated parameters
- LL: log-likelihood
- AICc
- Delta: ΔAICc, the difference between each model's AICc and the lowest AICc among the models
- Weight: AICc weight, the relative likelihood that each model is the best of the candidate models, calculated as

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum_j \exp(-\Delta_j/2)}$$

- Cum_Weight: cumulative weight

We also calculate AIC weights directly from each model's AIC, without the small-sample correction:

- Extracts AIC for each model using `AIC()`
- Computes ΔAIC as the difference from the lowest AIC
- Calculates AIC weights using the same formula as above

With large samples (840 observations in the example data), AIC and AICc are nearly identical, so both tables lead to the same conclusions.

Interpretation:

In most cases, the hyperbolic model will have the lowest AICc, so the exponential and linear models will have positive ΔAICc values. ΔAICc < 2 → substantial support (models are essentially equivalent) ΔAICc 4–7 → considerably less support ΔAICc > 10 → essentially no support (the model with the lower AICc fits better)

AICc weights sum to 1 across the models. An AICc weight of 1 for the hyperbolic model means it is overwhelmingly the best-fitting model compared to the exponential and linear alternatives.

```
models_base <- list(hyperbolic  = base_model_short,
                    exponential = exponential_model_base,
                    linear      = linear_model_base)

# AICc table (same values as aictab() in the AICcmodavg package)
AICc_table_base <- aic_table(models_base)
print(AICc_table_base)
```

```
##         Model K        LL   n     AICc    Delta        Weight Cum_Weight
## 1  hyperbolic 6 -1087.080 840 2186.262   0.0000  1.000000e+00          1
## 2 exponential 6 -1148.447 840 2308.995 122.7339  2.231921e-27          1
## 3      linear 6 -1415.699 840 2843.500 657.2383 1.916554e-143          1
```

```
# AIC weights (manual)
AICs_base <- sapply(models_base, AIC)
delta_base <- AICs_base - min(AICs_base)
weights_base <- exp(-0.5 * delta_base) / sum(exp(-0.5 * delta_base))


AIC_table_base <- data.frame(
  Model = names(AICs_base),
  AIC = as.numeric(AICs_base),
  Delta = delta_base,
  Weight = weights_base
)
print(AIC_table_base)
```

```
##                   Model      AIC    Delta        Weight
## hyperbolic   hyperbolic 2186.161   0.0000  1.000000e+00
## exponential exponential 2308.895 122.7339  2.231921e-27
## linear           linear 2843.399 657.2383 1.916554e-143
```

Extract logk

Below we extract individual logk values for each participant. In most cases, logk values should be extracted from the base model (Model 1). This ensures that individual discounting estimates are derived directly from task behavior and are not influenced by predictors included in the hyperbolic model. These participant-level logk values can then be used in subsequent analyses (e.g., regression models with covariates).

The code below combines the fixed effects and random effects to calculate participant-level logk estimates.

```
cf_short <- coef(base_model_short)                    # fixed + random, one row per sub
num
individual_logk_df_base_short <- data.frame(
  subnum    = type.convert(rownames(cf_short), as.is = TRUE),
  logk_base = cf_short$logk
)
head(individual_logk_df_base_short)
```

```
##   subnum logk_base
## 1      1 -3.719229
## 2      2 -1.682529
## 3      3 -2.494020
## 4      4 -5.821592
## 5      5 -3.936651
## 6      6 -1.597337
```

Check Against the True Values (Practice Data Only)

Same check for the short form, using `SDshortform_exampledata_trueparameters.csv` .

```
true_file_short <- file.path(path, sub("\\.csv$", "_trueparameters.csv", sd_short))

if (file.exists(true_file_short)) {
  true_short <- read.csv(true_file_short)

  check_short <- individual_logk_df_base_short %>%
    inner_join(true_short, by = "subnum")

  print(cor.test(check_short$logk, check_short$logk_base))
}
```

```
##
##  Pearson's product-moment correlation
##
## data:  check_short$logk and check_short$logk_base
## t = 32.767, df = 118, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.9278131 0.9643679
## sample estimates:
##       cor
## 0.9492001
```

Extracting AUC

Area under the curve (AUC) is computed using the trapezoidal method (trapz) across the observed social distance values (1, 2, 5, 10, 20, 50, 100). Because AUC is calculated over the actual range of x-values included in the task, the total span of the social distance axis is 100 − 1 = 99 units. The maximum possible AUC therefore corresponds to a participant consistently selecting the maximum forgone amount (10) across this range, yielding a value of 99 × 10 = 990. Observed AUC values were divided by 990 to yield a normalized index bounded between 0 and 1.

```
trial_filtered_data2 <- dplyr::left_join(
  trial_filtered_data_s,
  individual_logk_df_base_short,
  by = "subnum"
)


AUC_short <- trial_filtered_data2 %>%
  group_by(subnum) %>%
  summarise(AUC = trapz(as.numeric(Distance), as.numeric(max_rating))/990, .groups = "dr
op") %>%
  left_join(
    trial_filtered_data2 %>%
      dplyr::distinct(subnum, .keep_all = TRUE) %>%
      dplyr::select(-max_rating, -Distance, -any_of(c("distance", "trial"))),
    by = "subnum"
  )
```

Model 2: Model With Predictors (Optional)

You may want to examine how participant characteristics predict differences in the hyperbolic discounting function itself. This model allows predictors (e.g., demographic variables, personality traits, or group membership) to explain variability in the discounting rate (logk).

This step is **optional** and is typically used only if the goal is to test predictors **directly within the hyperbolic model**, rather than analyzing extracted logk values in a separate model.

Tips for using this model:

Modify the predictors in the example to match the variables available in your dataset.

Set `n_extra` in `fit_best_start()` to the number of predictors in the model (5 in this example). The starting values for the predictor slopes are set to 0.

```
pred_hyp_fit_s <- function(st) nlme(as.numeric(max_rating) ~ v0 / (1 + exp(logk) * Dista
nce),
                                    fixed     = list(v0 ~ 1, logk ~ as.numeric(Male) + a
s.numeric(age) + as.numeric(income) +
                                as.numeric(FIT_sum) + as.numeric(Group_var)),
                                    random    = v0 + logk ~ 1 | subnum,
                                    start     = st,
                                    data      = trial_filtered_data_s,
                                    control   = ctl,
                                    na.action = na.omit)

predictor_model <- fit_best_start(pred_hyp_fit_s, start_grid_s, n_extra = 5)
summary(predictor_model)
attr(predictor_model, "start_search")
```

Comparison Models

We compare the hyperbolic model with an exponential and a linear model to confirm that the hyperbolic model fits best.

- **Exponential model:** like the hyperbolic model, it is parameterized with logk (k = exp(logk)), which keeps k positive. Its starting values are found with `fit_best_start()`. Note that the exponential logk is not on the same scale as the hyperbolic logk, so the two should not be compared directly.
- **Linear model:** this model is linear, so it is fit with `lme()`, which does not need starting values. `method = "ML"` is required so that its AICc can be compared with the `nlme` models. In `lme()`, a predictor's effect on the slope is written as an interaction with Distance (`Distance:predictor`), so the predictors affect the rate of decline, as in the hyperbolic and exponential models.

```
# Exponential model
pred_exp_fit_s <- function(st) nlme(as.numeric(max_rating) ~ v0 * exp(-exp(logk) * Dista
nce),
                                    fixed     = list(v0 ~ 1, logk ~ as.numeric(Male) + a
s.numeric(age) + as.numeric(income) +
                                                                    as.numeric(FIT_sum) + as.numer
ic(Group_var)),
                                    random    = v0 + logk ~ 1 | subnum,
                                    start     = st,
                                    data      = trial_filtered_data_s,
                                    control   = ctl,
                                    na.action = na.omit)


exponential_model_pred <- fit_best_start(pred_exp_fit_s, start_grid_s, n_extra = 5)
summary(exponential_model_pred)

# Linear model
linear_model_pred <- lme(as.numeric(max_rating) ~ Distance +
                           Distance:(as.numeric(Male) + as.numeric(age) + as.numeric(inc
ome) +
          as.numeric(FIT_sum) + as.numeric(Group_var)),
                         random    = ~ Distance | subnum,
                         data      = trial_filtered_data_s,
                         method    = "ML",
                         na.action = na.omit,
                         control   = lmeControl(opt = "optim"))
summary(linear_model_pred)
```

Model Comparison (AICc Weights)

After fitting the hyperbolic, exponential, and linear models, we compare their relative fit using Akaike's Information Criterion corrected for sample size (AICc). AICc balances model fit and complexity: lower values indicate better-fitting models, with a penalty for additional parameters.

The `aic_table()` function (see Model Fitting Functions) returns one row per model, ranked from best to worst fit:

- K: number of estimated parameters
- LL: log-likelihood
- AICc
- Delta: ΔAICc, the difference between each model's AICc and the lowest AICc among the models
- Weight: AICc weight, the relative likelihood that each model is the best of the candidate models, calculated as

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum_j \exp(-\Delta_j/2)}$$

- Cum_Weight: cumulative weight

We also calculate AIC weights directly from each model's AIC, without the small-sample correction:

- Extracts AIC for each model using `AIC()`

- Computes ΔAIC as the difference from the lowest AIC
- Calculates AIC weights using the same formula as above

With large samples (840 observations in the example data), AIC and AICc are nearly identical, so both tables lead to the same conclusions.

Interpretation:

In most cases, the hyperbolic model will have the lowest AICc, so the exponential and linear models will have positive ΔAICc values. ΔAICc < 2 → substantial support (models are essentially equivalent) ΔAICc 4–7 → considerably less support ΔAICc > 10 → essentially no support (the model with the lower AICc fits better)

AICc weights sum to 1 across the models. An AICc weight of 1 for the hyperbolic model means it is overwhelmingly the best-fitting model compared to the exponential and linear alternatives.

```
models_pred <- list(hyperbolic  = predictor_model,
                    exponential = exponential_model_pred,
                    linear      = linear_model_pred)

# AICc table (same values as aictab() in the AICcmodavg package)
AICc_table_pred <- aic_table(models_pred)
AICc_table_pred

# AIC weights (manual)
AICs_pred <- sapply(models_pred, AIC)
delta_pred <- AICs_pred - min(AICs_pred)
weights_pred <- exp(-0.5 * delta_pred) / sum(exp(-0.5 * delta_pred))
weights_pred

AIC_table_pred <- data.frame(
  Model = names(AICs_pred),
  AIC = as.numeric(AICs_pred),
  Delta = delta_pred,
  Weight = weights_pred
)
AIC_table_pred
```

Extracting logk values

In the predictor model, each participant's logk depends on their predictor values. It is calculated as the participant-specific intercept (the fixed intercept plus that participant's random effect) plus each fixed slope multiplied by the participant's value on that predictor. The code below:

- uses coef() to get each participant's model coefficients. The logk intercept already includes the participant's random effect.
- selects one row per participant from the data and puts the rows in the same order as the model output. Participants the model excluded (e.g., because of missing covariates) are dropped.
- creates a predictor matrix with each participant's covariate values. The predictors must match those in the hyperbolic model, in the same order.
- multiplies each participant's predictor values by the corresponding coefficients and sums them to get that participant's logk
- creates a dataset (individual_logk_df_pred) with one row per participant, containing the participant ID and estimated logk.

```
cf  <- coef(predictor_model)
pp  <- trial_filtered_data_s %>% distinct(subnum, .keep_all = TRUE)
pp  <- pp[match(type.convert(rownames(cf), as.is = TRUE), pp$subnum), ] #puts the rows i
n the same order as the participants in cf. also drops anyone that the model left out (e
x: for missing a covariate)
X <- model.matrix(~ as.numeric(Male) + as.numeric(age) + as.numeric(income) +
                    as.numeric(FIT_sum) + as.numeric(Group_var),
                  data = pp) #builds a matrix

individual_logk_df_pred_short <- data.frame(
  subnum    = pp$subnum,
  logk_pred = rowSums(X * as.matrix(cf[, grep("^logk", names(cf))]))
)
```

Extracting AUC

```
if (exists("AUC_short")) {
  AUC_short <- dplyr::left_join(
    AUC_short,
    individual_logk_df_pred_short,
    by = "subnum"
  )
} else {
  trial_filtered_data3 <- dplyr::left_join(
    trial_filtered_data_s,
    individual_logk_df_pred_short,
    by = "subnum"
  )

  AUC_short <- trial_filtered_data3 %>%
    group_by(subnum) %>%
    summarise(AUC = trapz(as.numeric(Distance), as.numeric(max_rating))/990, .groups =
"drop") %>%
    left_join(
      trial_filtered_data3 %>%
        dplyr::distinct(subnum, .keep_all = TRUE) %>%
        dplyr::select(-max_rating, -Distance, -any_of(c("distance","amount1","choice","t
rial"))),
      by = "subnum"
    )
}
```

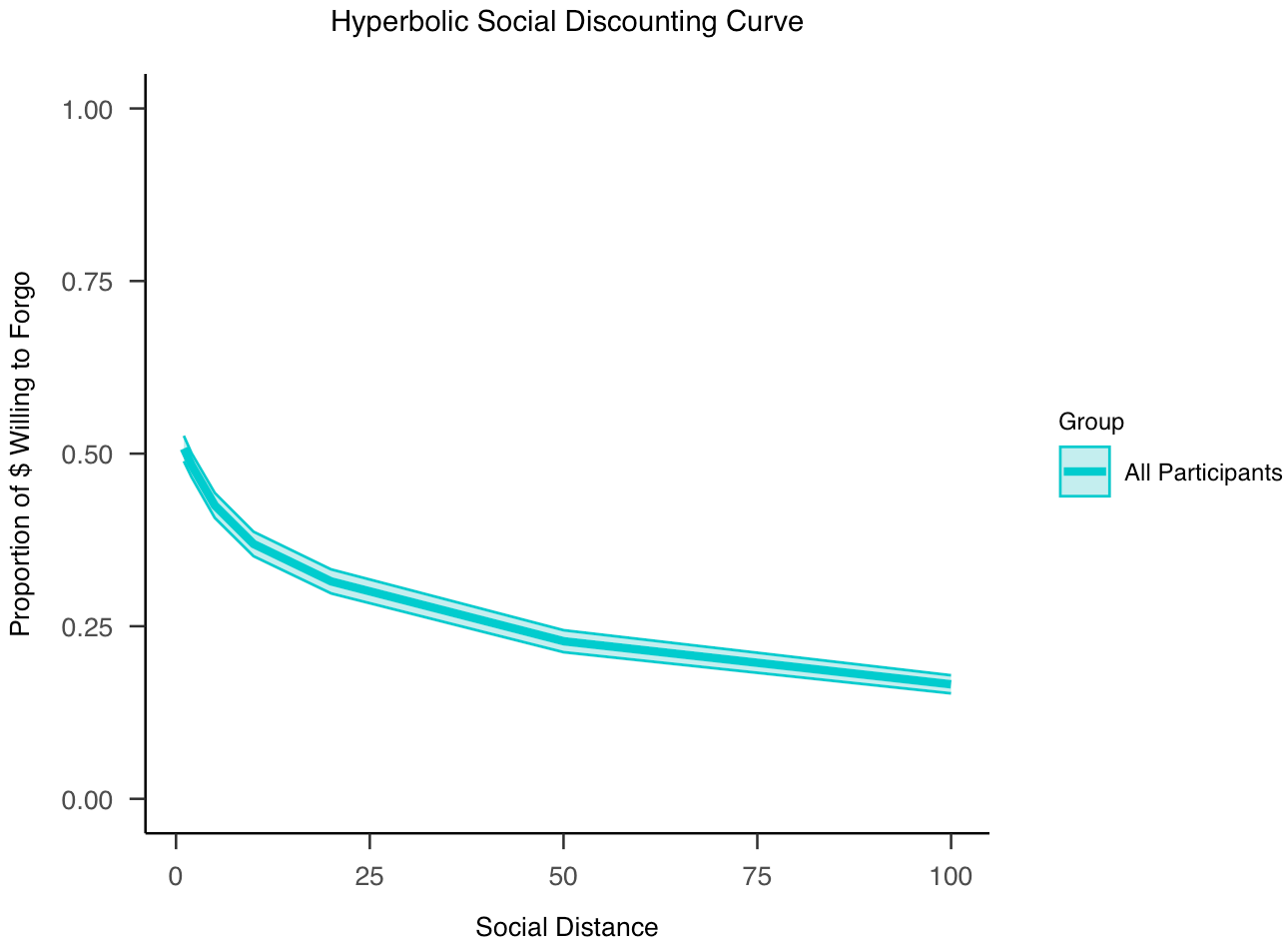
Graph of Social Discounting curve

This code generates a plot of participants' generosity across social distance, scaled as a proportion of the maximum possible amount (percentage). The workflow automatically adapts based on whether the dataset contains a grouping variable (Group):

- With "Group_var" present: The data are summarized separately for each group. The plot displays one line per group with shaded ribbons representing ±1 standard error of the mean. Groups are automatically labeled "Group1" and "Group2," and color-coded for clarity.

- Without “Group_var”: The data are summarized across all participants, producing a single hyperbolic curve with its standard error ribbon.

This approach ensures the graph works whether or not multiple groups are present, providing a clear visualization of social discounting behavior in your sample.

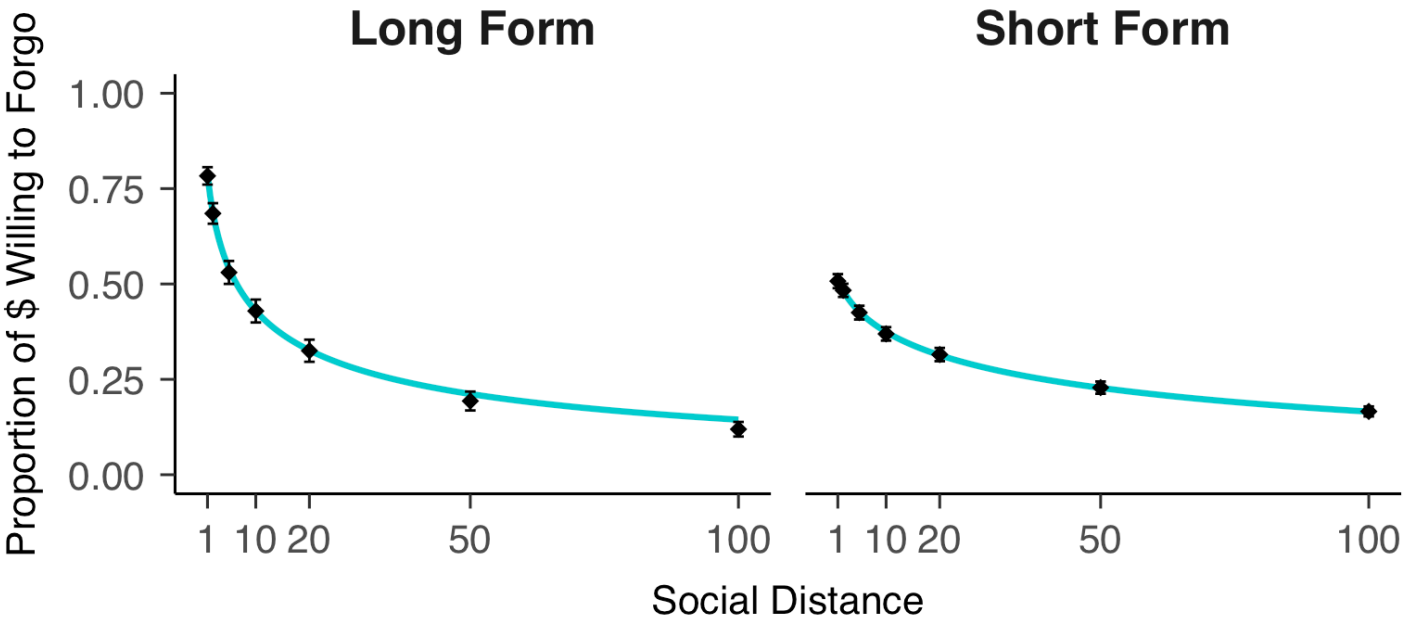


Graph of Both Forms Together

This code puts the long form (left) and short form (right) in one image:

- **Line:** the fitted hyperbolic curve, averaged across participants (each participant’s curve uses their own v0 and logk from the base model)
- **Diamonds:** the actual mean proportion forgone at each social distance, with error bars showing ±1 standard error

If “Group_var” is present, each group gets its own line and diamonds. The image is also saved as a PNG in the folder set in `out_dir`.


Long Form
Short Form
Proportion of $ Willing to Forgo
1.00
0.75
0.50
0.25
0.00
1
10
20
50
100
Social Distance